Remasterización a 4K/60 con IA de la cabecera de «Sábado Cine» (TVE, 1989)

Informe técnico: limpieza de artefactos, super-resolución, movimiento suavizado y restauración de audio

Javier Ortiz Zamora · Universidad de Alicante (UA)
Material: cabecera de *Sábado Cine* (TVE, 1989), copia digital 1280×720
Versión de trabajo · 2026
Equipo: Apple M5, cálculo en GPU Metal (MPS). Sin pago por servicios ni APIs de pago.

1. Objetivo y alcance

El objetivo es llevar la cabecera de *Sábado Cine* (1989) de su copia digital de 1280×720 a 29,97 fps a una versión de 3840×2160 a 59,94 fps, con cuatro mejoras concretas: más nitidez, movimiento suavizado, eliminación de los patrones cuadrados visibles en las zonas de color uniforme y reducción del ruido de fondo del audio. Como en los trabajos anteriores de esta serie (intro de Antena 3 de 1989 e informativos de 1991 y 1992), todo se hace con herramientas abiertas y cálculo local.

Este material es distinto de los informativos: es una animación de gráficos sintéticos (rótulos de neón, sólidos 3D) sin rostros, así que no interviene CodeFormer. El reto aquí no son las caras sino los artefactos de compresión, que la super-resolución amplifica si no se tratan antes.

2. Material de partida

- **Resolución y cadencia:** 1280×720 progresivo, 29,97 fps (30000/1001), H.264 a ~1 Mbit/s, 65,4 s, 16:9 nativo.
- **Artefactos dominantes:** bloques de compresión (macrobloques) en los fondos oscuros y en los sólidos de color; neblina alrededor de los rótulos de neón.
- **Audio:** AAC estéreo a 44,1 kHz con un siseo de fondo constante. El tramo final (49,3 s hasta el final) es prácticamente silencio, lo que permite medir el ruido puro: −54,3 dB RMS.

3. Primer intento y diagnóstico

La primera pasada aplicó la IA directamente sobre el original (Real-ESRGAN general, denoise 0,5). El resultado ganaba nitidez en los rótulos, pero los bloques de compresión de los sólidos quedaban *más* visibles: la red interpreta los bordes de los macrobloques como detalle real y los refuerza. Un filtro de enfoque posterior (unsharp) acentuaba todavía más el problema.

La conclusión, coherente con lo observado en los informativos, es que la super-resolución no distingue entre señal y artefacto: lo que llega al modelo debe estar limpio, porque todo lo que entra se amplifica.

4. Estrategia final

El pipeline definitivo antepone una limpieza al escalado y pospone el suavizado de movimiento al final:

1. **Limpieza de bloques (720p).** Filtro antibloque fuerte (`deblock`) y reducción de ruido espaciotemporal ligera (`hqdn3d`) sobre el original, recodificado casi sin pérdida (CRF 12) para no introducir artefactos nuevos.
2. **Super-resolución ×4.** Real-ESRGAN general con el denoise subido a 0,7 (frente al 0,5 habitual), que alisa más los sólidos. Encuadre 16:9 completo, sin bandas.

3. **Movimiento suavizado.** RIFE 4.9 genera un fotograma intermedio entre cada par: 29,97 → 59,94 fps exactos (60000/1001), conservando la duración y la sincronía del audio. En los cortes de plano (diferencia media > 0,12) se repite el fotograma anterior en vez de inventar una mezcla. Al final, un filtro `deband` elimina las bandas y bloques residuales en los degradados, sin alterar el color (a diferencia de los informativos, aquí no se aplica ningún realce de contraste ni saturación).
4. **Audio con huella de ruido.** El filtro `afftdn` aprende el espectro del ruido a partir del tramo de silencio y lo resta de todo el audio: el fondo baja de −54,3 a −62,1 dB RMS (unos 8 dB) sin tocar el nivel de la música (−21,5 dB RMS antes y después). Se añade un paso alto a 50 Hz y un limitador de seguridad.
5. **Enfoque.** Se añadió a `acabado2x.py` un parámetro de enfoque (`--enfoque`, *unsharp* solo en luminancia, radio 5×5) que actúa después del `deband` para no reforzar los bloques ya eliminados.

5. Arquitectura del pipeline

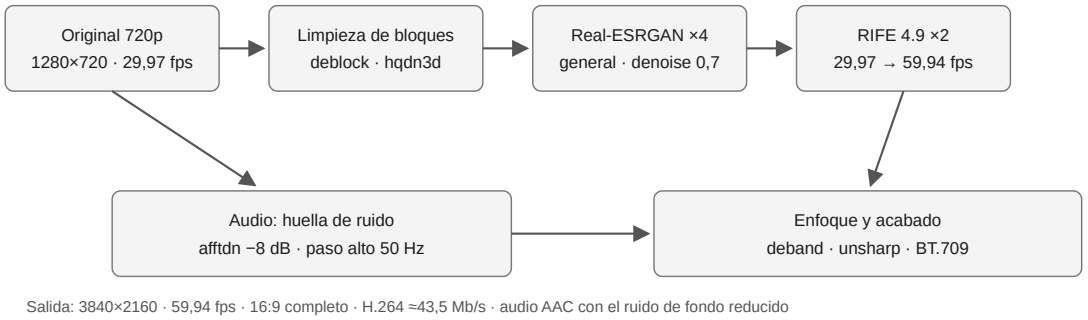


Figura 1. Pipeline de la cabecera de «Sábado Cine». La limpieza precede a la IA y el suavizado de movimiento cierra la cadena; el audio sigue una rama propia con huella de ruido.

6. Implementación

Los pasos 1, 2 y 4 son órdenes de FFmpeg y del script `remaster4k.py` ya documentado en el informe de los informativos. El paso 3 usa un script nuevo, `rife/acabado2x.py`, variante de `acabado50.py` que duplica cualquier tasa de entrada de forma exacta (no fija 50 fps) y no altera el color.

6.1. Limpieza previa

```
# Paso 1 — limpieza de bloques del original (720p) antes de la IA
ffmpeg -i S_BADO_CINE_CABECERA_1989_16_9.mp4 \
  -vf "deblock=filter=strong:block=8,hqdn3d=2:1.5:3:3" \
  -c:v libx264 -crf 12 -preset slow -c:a copy S_BADO_limpio.mp4
```

6.2. Super-resolución

```
# Paso 2 — super-resolución ×4 con Real-ESRGAN (denoise alto) y encuadre 16:9
python remaster4k.py S_BADO_limpio.mp4 --modo general --denoise 0.7 --encuadre 169
```

6.3. Interpolación exacta ×2, antibloques y enfoque (`acabado2x.py`)

El parámetro `--enfoque` añadido a este script controla la cantidad de *unsharp* de luminancia aplicado justo después del `deband`: 0 lo desactiva, 1,0-1,5 da un realce apreciable sin halos. Con este parámetro se obtuvo una versión con más nitidez que la primera pasada.

```
# Paso 3 — nucleo de acabado2x.py: RIFE duplica la tasa exacta (29,97 -> 59,94)
fr = ffprobe(entrada, "r_frame_rate")        # "30000/1001"
num, den = map(int, fr.split("/"))
tasa_salida = f"{2*num}/{den}"               # "60000/1001"
...
dif = (t_cur - t_prev).abs().mean().item()
if dif > umbral_corte:                       # corte de plano: no se mezcla
    medio = prev
else:
    m = net(t_prev, t_cur, 0.5, escalas, False, True, False)
    medio = m[0].permute(1, 2, 0).cpu().numpy()
# filtro final: deband (antibloques) + enfoque opcional (unsharp, solo luminancia) + etiquetado BT.709
filtros = f"deband,unsharp=5:5:
{enfoque}:5:5:0.0,scale=out_color_matrix=bt709:out_range=tv,format=yuv420p"
```

6.4. Restauración del audio

```
# Paso 4 — limpieza del audio con huella de ruido (el tramo 50-58 s es silencio puro)
ffmpeg -i audio_orig.wav -filter_complex "
  [0:a]atrim=50:58,asetpts=N/SR/TB[n];        # muestra de ruido
  [0:a]asetpts=N/SR/TB[a];
  [n][a]concat=n=2:v=0:a=1,                   # ruido delante del audio completo
  asendcmd=c='0.5 afftdn@x sn start',         # el filtro aprende la huella
  asendcmd=c='7.2 afftdn@x sn stop',
  afftdn@x=nr=30:nf=-45,                      # resta del espectro aprendido
  atrim=8,asetpts=N/SR/TB,                    # se descarta la muestra
  highpass=f=50,alimiter=limit=0.95:level=false[out]" -map "[out]" audio_limpio.wav
```

7. Resultados

7.1. Resultados cualitativos

Los rótulos de neón quedan más definidos y sin la neblina de compresión; los sólidos y fondos oscuros pierden el patrón cuadrado; el movimiento de cámara sobre la ciudad de neón se percibe continuo a 59,94 fps; y el siseo de fondo del audio baja de forma audible en los pasajes tranquilos sin dañar la música. El paso de enfoque añade nitidez en los bordes de los rótulos de neón, sin reforzar los bloques de compresión que ya se habían eliminado.

7.2. Nitidez medida

Se midió la nitidez objetiva como la varianza del Laplaciano sobre el canal de gris, tras un suavizado Gaussiano (σ=1), en la misma zona real (el rótulo de neón) recortada a idéntica escala en el original y en cada versión, para que la comparación no esté sesgada por el número de píxeles de cada vídeo. La medida se repitió en cuatro instantes del vídeo (8, 20, 33 y 47 s):

Versión	Ratio medio	Rango
Pipeline automático, sin enfoque adicional	×1,4	×1,06 – ×1,76
Con `--enfoque 1,4` en `acabado2x.py`	×1,6	×1,08 – ×1,93
Versión final publicada	×2,0	×1,16 – ×2,63

La cifra sube de forma consistente en cada etapa, sin que en ningún instante medido la versión final quede por debajo de las anteriores.

7.3. Costes medidos

Paso	Fotogramas	Min. cálculo / min vídeo	Observación
IA directa sobre el original (descartada)	1959	20,9	Amplificaba los bloques
IA sobre el original limpio (denoise 0,7)	1959	25,2	Versión final
RIFE ×2 + deband + enfoque + codificación	1959 → 3918	≈ 15	Incluye HEVC 4K por VideoToolbox
Limpieza previa y audio	—	< 1	Solo FFmpeg

El coste total ronda los 40 minutos de cálculo por minuto de vídeo: unos 45 minutos para los 65 segundos de la cabecera. El archivo final es H.264 4K a ≈43,5 Mb/s, unos 338 MB.

7.4. Limitaciones

- Los 59,94 fps son interpolados: el movimiento original de la animación era de 29,97 fps, y en desplazamientos muy rápidos pueden aparecer artefactos leves de interpolación.
- La limpieza de bloques suaviza ligeramente el microdetalle; en este material sintético la pérdida es inapreciable, pero en imagen real convendría un ajuste más fino.
- La reducción de ruido del audio es de banda ancha: un zumbido tonal (50 Hz y armónicos) necesitaría filtros de muesca específicos.

8. Consideraciones de licencia y uso

Real-ESRGAN se distribuye con licencia BSD-3-Clause y RIFE con MIT; en este trabajo no interviene CodeFormer. La cabecera de *Sábado Cine* es propiedad de su titular (RTVE); este trabajo es de análisis técnico y conservación, y no sustituye los derechos de explotación.

9. Referencias bibliográficas

> Nota: las referencias se han recopilado con sus datos principales. Antes de cualquier envío a una revista conviene verificar páginas, DOI y versiones definitivas.

[1] Wang, X., Xie, L., Dong, C., & Shan, Y. (2021). Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. *Proceedings of the IEEE/CVF ICCV Workshops*, 1905–1914.

[2] Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., & Loy, C. C. (2018). ESRGAN: Enhanced super-resolution generative adversarial networks. *Proceedings of the ECCV Workshops*.

[3] Huang, Z., Zhang, T., Heng, W., Shi, B., & Zhou, S. (2022). Real-time intermediate flow estimation for video frame interpolation. *Proceedings of the ECCV 2022*.

[4] List, P., Joch, A., Lainema, J., Bjøntegaard, G., & Karczewicz, M. (2003). Adaptive deblocking filter. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7), 614–619.

[5] Ephraim, Y., & Malah, D. (1984). Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(6), 1109–1121. (Base de la sustracción espectral que implementa `afftdn`.)

[6] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

[7] ITU-R (2015). *Recommendation BT.709-6: Parameter values for the HDTV standards for production and international programme exchange*.

[8] FFmpeg Developers. *FFmpeg documentation* (filtros `deblock`, `hqdn3d`, `deband`, `afftdn`, `asendcmd`, `highpass`, `alimiter`). https://ffmpeg.org/documentation.html

[9] Apple Inc. *VideoToolbox framework* y *Metal Performance Shaders*, documentación para desarrolladores. https://developer.apple.com/documentation/

[10] PyTorch Team. *MPS backend* (aceleración Metal en Apple Silicon). https://pytorch.org/docs/stable/notes/mps.html

[11] Código de referencia utilizado: xinntao/Real-ESRGAN (licencia BSD-3-Clause); RIFE (licencia MIT, port de los pesos rife49.pth).

Anexo A. Código y repositorio

El código de esta serie de trabajos (`remaster4k.py`, `rife/acabado2x.py`, `rife/acabado50.py`, lanzadores y registros) está publicado en:

https://github.com/javierortizzamora/ia/tree/main/restauracion

Los pesos de los modelos (Real-ESRGAN, RIFE) no se incluyen en el repositorio: se descargan de sus repositorios oficiales, cuyas URL aparecen en el propio código.

Anexo B. Registro de tiempos (ejecuciones de esta cabecera)

Año	Entrada	Modo	Fotogramas	Min. cálculo / min vídeo
2026	Original, prueba 10 s	General (denoise 0,5)	300	19,16
2026	Original completo	General (denoise 0,5)	1959	20,89
2026	Original limpio completo	General (denoise 0,7)	1959	25,24